

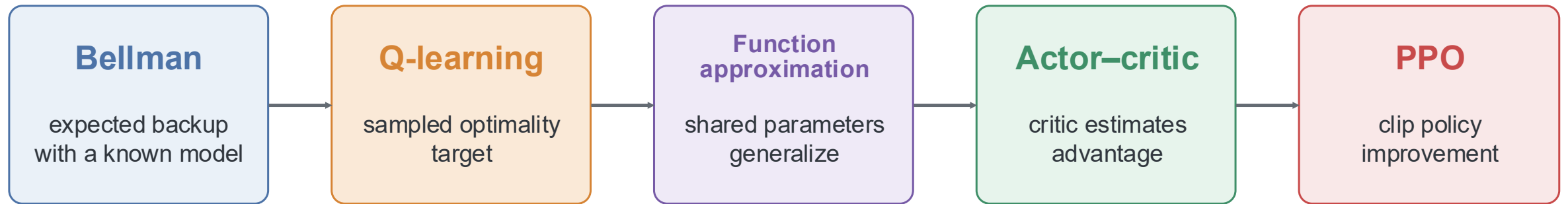


Tutorial 4

Function Approximation and Policy Search

Bellman equations · Q-learning · Function approximation · PPO

■ One learning target, four increasingly scalable choices



target – current estimate → update

The objects change. The learning pattern stays recognizable.



01

Bellman to Q-learning

Expected backups when the model is known, sampled backups when it is not

RECALL · DISCUSS · CALCULATE

Recap: MDP

Problem Formulation

As in Chapter 3, the **transition model** (or just “model,” when the meaning is clear) describes the outcome of each action in each state. Here, the outcome is stochastic, so we write $P(s' | s, a)$ for the probability of reaching state s' if action a is done in state s . (Some authors write $T(s, a, s')$ for the transition model.) We will assume that transitions are **Markovian**: the probability of reaching s' from s depends only on s and not on the history of earlier states.

$$P(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(s_{t+1} | s_t, a_t)$$

Later in this section, we investigate the nature of utility functions on histories; for now, we simply stipulate that for every transition from s to s' via action a , the agent receives a **reward** $R(s, a, s')$. The rewards may be positive or negative, but they are bounded by $\pm R_{\max}$.¹

Solution

goal. Therefore, a solution must specify what the agent should do for *any* state that the agent might reach. A solution of this kind is called a **policy**. It is traditional to denote a policy by π , and $\pi(s)$ is the action recommended by the policy π for state s . No matter what the outcome

Recap: Utilities

assume the agent is in some initial state s and define S_t (a random variable) to be the state the agent reaches at time t when executing a particular policy π . (Obviously, $S_0 = s$, the state the agent is in now.) The probability distribution over state sequences $S_1, S_2, \dots$, is determined by the initial state s , the policy π , and the transition model for the environment.

The expected utility obtained by executing π starting in s is given by

$$U^\pi(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1}) \right], \quad (17.2)$$

The utility function $U(s)$ allows the agent to select actions by using the principle of maximum expected utility from Chapter 16—that is, choose the action that maximizes the reward for the next step plus the expected discounted utility of the subsequent state:

$$\pi^*(s) = \operatorname{argmax}_{a \in A(s)} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')]. \quad (17.4)$$

of a state and the utility of its neighbors: *the utility of a state is the expected reward for the next transition plus the discounted utility of the next state, assuming that the agent chooses the optimal action.* That is, the utility of a state is given by

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')]. \quad (17.5)$$

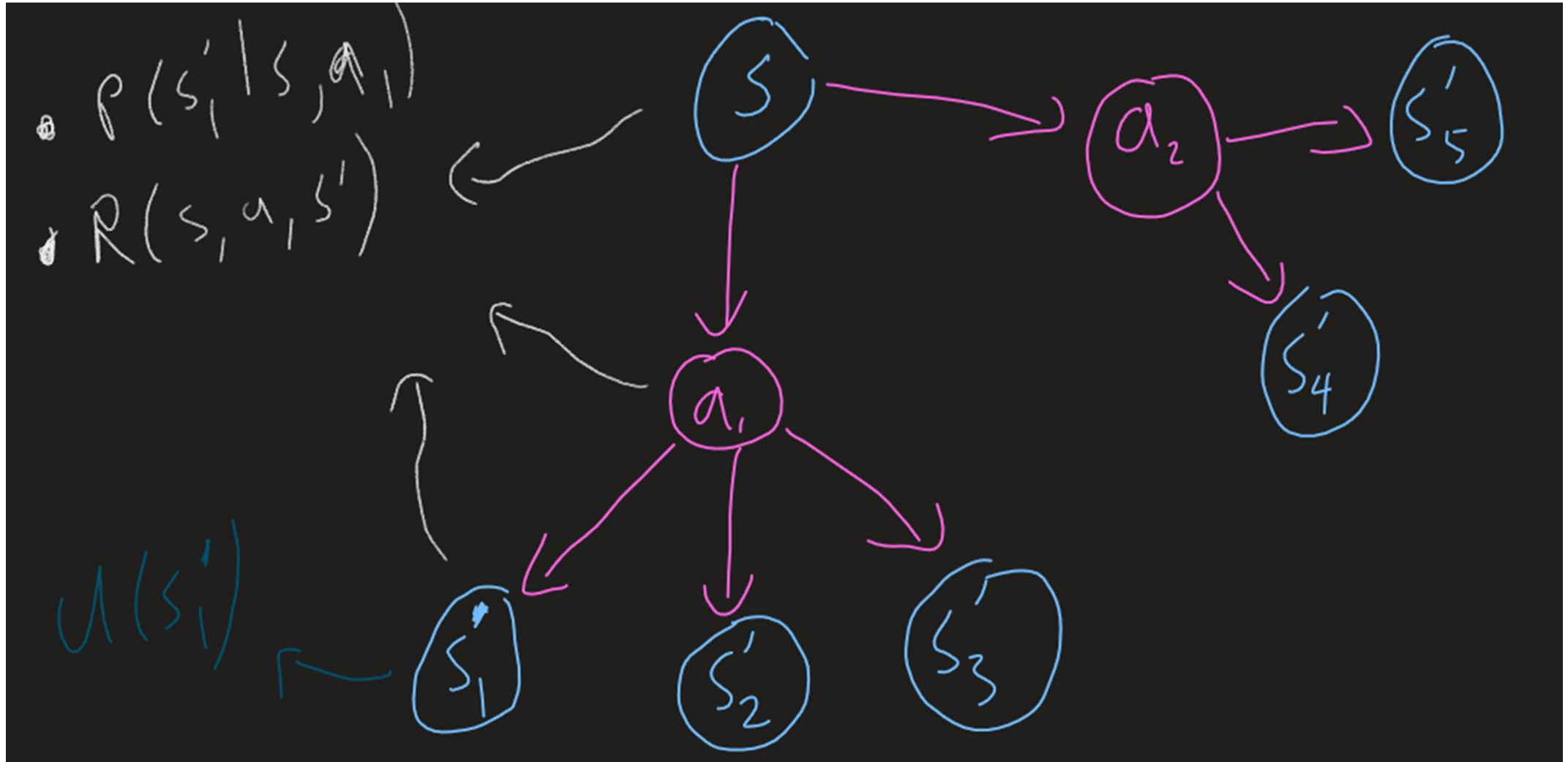
This is called the **Bellman equation**, after Richard Bellman (1957). The utilities of the

Recap: Utilities

$$\pi^*(s) = \operatorname{argmax}_{a \in A(s)} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')].$$

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')].$$

This is called the **Bellman equation**, after Richard Bellman (1957).



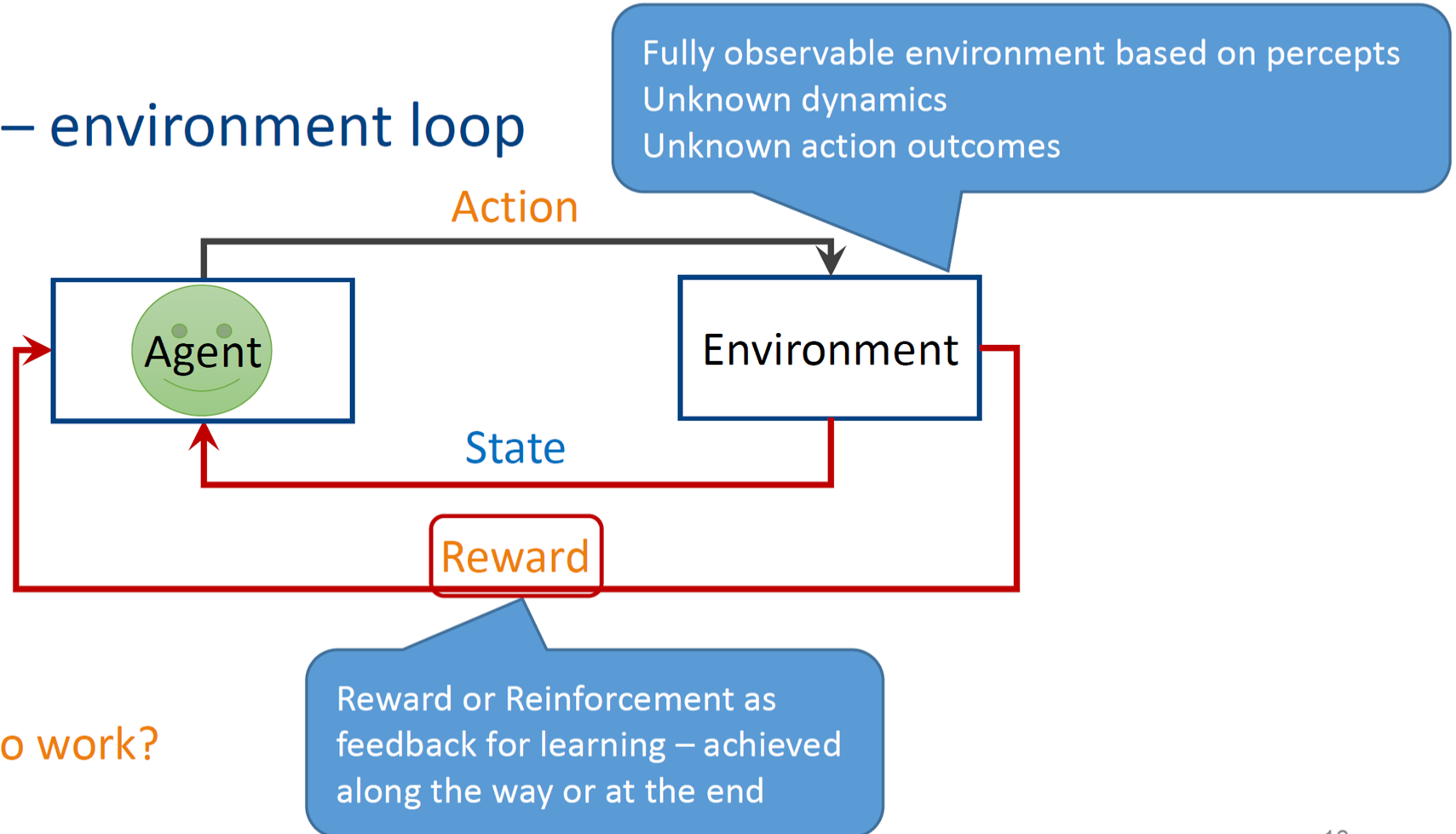
RECAP: Why RL? RL v.s. MDP?

maximize the expected sum of rewards. Reinforcement learning differs from “just solving an MDP” because the agent is not *given* the MDP as a problem to solve; the agent is *in* the MDP. It may not know the transition model or the reward function, and it has to act in order to learn more. Imagine playing a new game whose rules you don’t know; after a hundred or so moves, the referee tells you “You lose.” That is reinforcement learning in a nutshell.

RECAP: Why RL? RL v.s. MDP?

Reinforcement Learning

- The agent – environment loop



Example:
How does AlphaGo work?

RECAP: Why RL? RL v.s. MDP?

Learning Utility Function

Unknown transition dynamics, unknown reward functions.

- Model-based

- Learn the transition and reward functions, then solve the MDP (U^π)
- Examples: Adaptive Dynamic Programming

- Model-free

- Directly learn the utility function U^π
- Examples: Monte Carlo Learning, TD Learning

RECAP: Direct Utility Estimation

Trial 1	$(1, 1) \xrightarrow[\text{Up}]{-.04} (1, 2) \xrightarrow[\text{Up}]{-.04} (1, 3) \xrightarrow[\text{Right}]{-.04} (1, 2) \xrightarrow[\text{Up}]{-.04} (1, 3) \xrightarrow[\text{Right}]{-.04} (2, 3) \xrightarrow[\text{Right}]{-.04} (3, 3) \xrightarrow[\text{Right}]{+1} (4, 3)$
Trial 2	$(1, 1) \xrightarrow[\text{Up}]{-.04} (1, 2) \xrightarrow[\text{Up}]{-.04} (1, 3) \xrightarrow[\text{Right}]{-.04} (2, 3) \xrightarrow[\text{Right}]{-.04} (3, 3) \xrightarrow[\text{Right}]{-.04} (3, 2) \xrightarrow[\text{Up}]{-.04} (3, 3) \xrightarrow[\text{Right}]{+1} (4, 3)$
Trial 3	$(1, 1) \xrightarrow[\text{Up}]{-.04} (1, 2) \xrightarrow[\text{Up}]{-.04} (1, 3) \xrightarrow[\text{Right}]{-.04} (2, 3) \xrightarrow[\text{Right}]{-.04} (3, 3) \xrightarrow[\text{Right}]{-.04} (3, 2) \xrightarrow[\text{Up}]{-1} (4, 2)$

$$U^\pi(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1}) \right]$$

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')]$$

22.2.1 Direct utility estimation

The idea of **direct utility estimation** is that the utility of a state is defined as the expected total reward from that state onward (called the expected **reward-to-go**), and that each trial provides a *sample* of this quantity for each state visited. For example, the first of the three trials shown earlier provides a sample total reward of 0.76 for state (1,1), two samples of 0.80 and 0.88 for (1,2), two samples of 0.84 and 0.92 for (1,3), and so on. Thus, at the end of each sequence, the algorithm calculates the observed reward-to-go for each state and updates the estimated utility for that state accordingly, just by keeping a running average for each state in a table. In the limit of infinitely many trials, the sample average will converge to the true expectation in Equation (22.1).

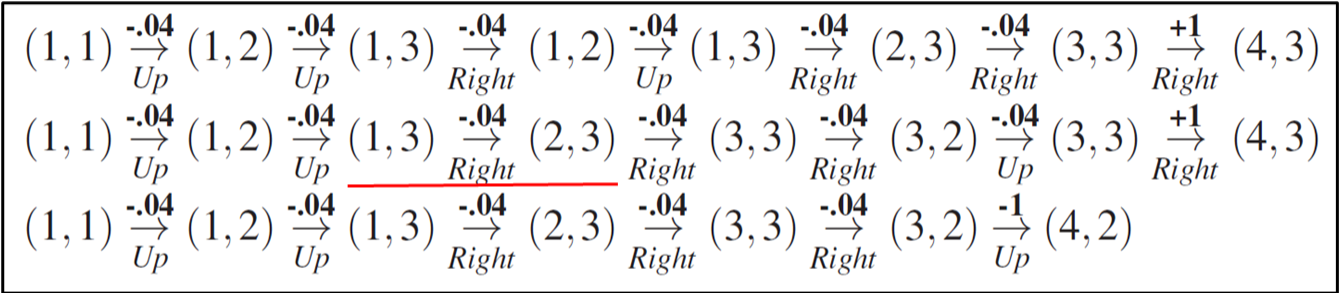
RECAP: ADP

$$\begin{array}{l}
 (1,1) \xrightarrow[\text{Up}]{-.04} (1,2) \xrightarrow[\text{Up}]{-.04} (1,3) \xrightarrow[\text{Right}]{-.04} (1,2) \xrightarrow[\text{Up}]{-.04} (1,3) \xrightarrow[\text{Right}]{-.04} (2,3) \xrightarrow[\text{Right}]{-.04} (3,3) \xrightarrow[\text{Right}]{+1} (4,3) \\
 (1,1) \xrightarrow[\text{Up}]{-.04} (1,2) \xrightarrow[\text{Up}]{-.04} (1,3) \xrightarrow[\text{Right}]{-.04} (2,3) \xrightarrow[\text{Right}]{-.04} (3,3) \xrightarrow[\text{Right}]{-.04} (3,2) \xrightarrow[\text{Up}]{-.04} (3,3) \xrightarrow[\text{Right}]{+1} (4,3) \\
 (1,1) \xrightarrow[\text{Up}]{-.04} (1,2) \xrightarrow[\text{Up}]{-.04} (1,3) \xrightarrow[\text{Right}]{-.04} (2,3) \xrightarrow[\text{Right}]{-.04} (3,3) \xrightarrow[\text{Right}]{-.04} (3,2) \xrightarrow[\text{Up}]{-1} (4,2)
 \end{array}$$

$$U_i(s) = \sum_{s'} P(s' | s, \pi_i(s)) [R(s, \pi_i(s), s') + \gamma U_i(s')].$$

Learning the transition model is easy, because the environment is fully observable. This means that we have a supervised learning task where the input for each training example is a state–action pair, (s, a) , and the output is the resulting state, s' . The transition model $P(s' | s, a)$ is represented as a table and it is estimated directly from the counts that are accumulated in $N_{s'|sa}$. The counts record how often state s' is reached when executing a in s . For example, in the three trials given on page 792, *Right* is executed four times in $(3,3)$ and the resulting state is $(3,2)$ twice and $(4,3)$ twice, so $P((3,2) | (3,3), \text{Right})$ and $P((4,3) | (3,3), \text{Right})$ are both estimated to be $\frac{1}{2}$.

RECAP: TD-Learning



RECAP: TD-Learning

Bellman Equation (Value/Policy Iteration)

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')].$$

TD Learning

$$U^\pi(s) \leftarrow \underbrace{U^\pi(s)}_{\text{Parameter to update}} + \alpha [\underbrace{R(s, \pi(s), s') + \gamma U^\pi(s')}_{\text{Target "Label"}} - \underbrace{U^\pi(s)}_{\text{"Current Est"}}] \quad \text{Error}$$

Parameter to update

Target "Label"

=

"Current Est"

=

Error

RECAP: Q Learning

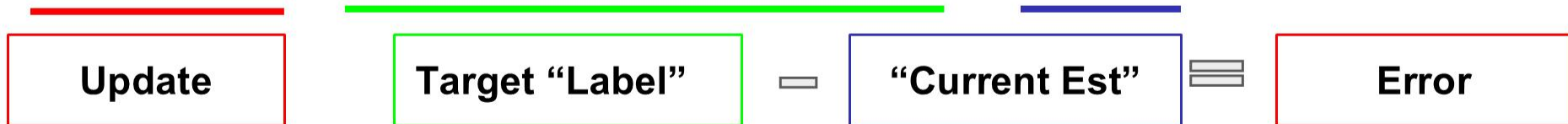
The **Q-learning** method avoids the need for a model by learning an action-utility function $Q(s, a)$ instead of a utility function $U(s)$. $Q(s, a)$ denotes the expected total discounted reward if the agent takes action a in s and acts optimally thereafter. Knowing the Q-function enables the agent to act optimally simply by choosing $\arg \max_a Q(s, a)$, with no need for look-ahead based on a transition model.

Bellman Equation (Value/Policy Iteration)

$$Q(s, a) = \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma \max_{a'} Q(s', a')] \quad (22.6)$$

TD Learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R(s, a, s') + \gamma \max_{a'} Q(s', a') - Q(s, a)]. \quad (22.7)$$



RECAP: SARSA

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R(s, a, s') + \gamma \max_{a'} Q(s', a') - Q(s, a)]. \quad \text{Q Learning} \quad (22.7)$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R(s, a, s') + \gamma Q(s', a) - Q(s, a)], \quad \text{SARSA} \quad (22.8)$$

The rule is applied at the end of each s, a, r, s', a' quintuplet—hence the name. The difference from Q-learning is quite subtle: whereas Q-learning backs up the Q-value from the best action in s' , SARSA waits until an action is taken and backs up the Q-value for that action.

Q-learning is an **off-policy** learning algorithm, because it learns Q-values that answer the question “What would this action be worth in this state, assuming that I stop using whatever policy I am using now, and start acting according to a policy that chooses the best action (according to my estimates)?” SARSA is an **on-policy** algorithm: it learns Q-values that answer the question “What would this action be worth in this state, assuming I stick with my policy?” Q-learning is more flexible than SARSA, in the sense that a Q-learning agent can learn how to behave well when under the control of a wide variety of exploration policies. On the other hand, SARSA is appropriate if the overall policy is even partly controlled by other agents or programs, in which case it is better to learn a Q-function for what will actually happen rather than what would happen if the agent got to pick estimated best actions. Both

■ Bellman, TD, and Q-learning differ mainly in the target

Known model

$$U(s) \leftarrow \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma U(s')]$$

One observed transition

$$\text{TD target: } y = r + \gamma \hat{U}(s')$$

$$\text{Q target: } y = r + \gamma \max_{a'} Q(s',a')$$

$$\text{estimate} \leftarrow \text{estimate} + \alpha [y - \text{estimate}]$$

Q-learning is off-policy because its target uses the greedy next action even when the behavior policy explores.



02 Function approximation

Replace one parameter per state with shared parameters that generalize across states

RECALL · DISCUSS · CALCULATE

Scaling: The Problem

- Number of states grow exponentially with # variables (features)
- Tabular representation (for utility function and Q -function) scales only to tens of thousands of states
- In comparison:
 - Backgammon: $\sim 10^{20}$
 - Chess: $\sim 10^{40}$
 - Go: $\sim 10^{172}$
 - Still considered relatively small as compared to real-world problems!
 - Cannot visit all the states infinitely often

Function Approximation: The Idea

Suppose that we want to build an agent that could estimate the **value** of a house **based on its size**.

- Approach 1:

- Collect data, put in a table (e.g., Excel sheet)
- If asked, then look up the answer
- This is what we were doing last week

- Better approach?

- Collect data, learn a function that fits the data
- Benefits: compact representation, generalization

1. Review

(a) Linear regression and loss function optimization.

Prediction: $h_{\mathbf{w}}(x) = w_1x + w_0$.

$$Loss(h_{\mathbf{w}}) = \sum_{j=1}^N L_2(y_j, h_{\mathbf{w}}(x_j)) = \sum_{j=1}^N (y_j - h_{\mathbf{w}}(x_j))^2 = \sum_{j=1}^N (y_j - (w_1x_j + w_0))^2.$$

$$\frac{\partial}{\partial w_i} Loss(\mathbf{w}) = \frac{\partial}{\partial w_i} (y - h_{\mathbf{w}}(x))^2$$

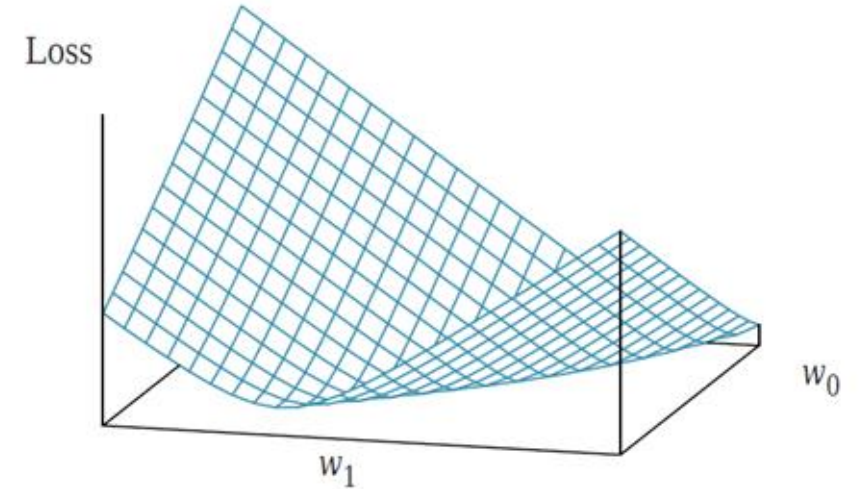
$$dL/dw = dL/dh * dh/dw$$

$$\frac{\partial}{\partial w_0} Loss(\mathbf{w}) = -2(y - h_{\mathbf{w}}(x)) \quad \frac{\partial}{\partial w_1} Loss(\mathbf{w}) = -2(y - h_{\mathbf{w}}(x)) \times x$$

$$w_0 \leftarrow w_0 + \alpha(y - h_{\mathbf{w}}(x)); \quad w_1 \leftarrow w_1 + \alpha(y - h_{\mathbf{w}}(x)) \times x.$$

Gradient Descent:

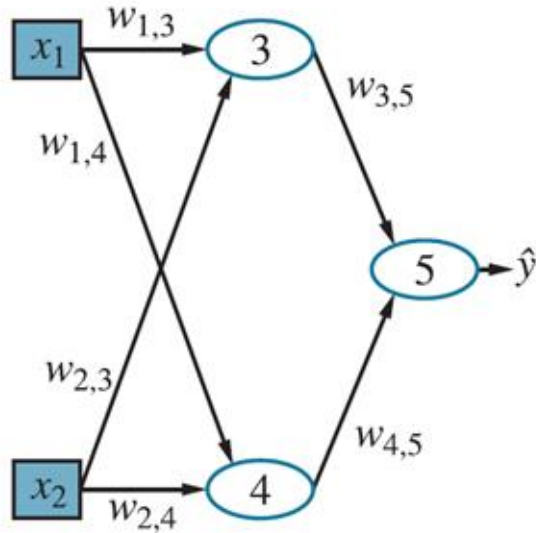
Repeat until converge {
 $w = w - \alpha \left[\frac{\partial Loss}{\partial w} \right]$
 $b = b - \alpha \left[\frac{\partial Loss}{\partial b} \right]$
}



Chain Rule: $\frac{d}{dx} [f(g(x))] = f'(g(x))g'(x)$

1. Review

(c) Deep learning in neural networks.



21.1 Simple Feedforward Networks

A **feedforward network**, as the name suggests, has connections only in one direction—that is, it forms a directed acyclic graph with designated input and output nodes. Each node computes a function of its inputs and passes the result to its successors in the network. Information flows through the network from the input nodes to the output nodes, and there are no loops.

21.1.1 Networks as complex functions

Each node within a network is called a **unit**. Traditionally, following the design proposed by McCulloch and Pitts, a unit calculates the weighted sum of the inputs from predecessor nodes and then applies a nonlinear function to produce its output. Let a_j denote the output of unit j and let $w_{i,j}$ be the weight attached to the link from unit i to unit j ; then we have

$$a_j = g_j(\sum_i w_{i,j} a_i) \equiv g_j(in_j),$$

where g_j is a nonlinear **activation function** associated with unit j and in_j is the weighted sum of the inputs to unit j .

As in Section 19.6.3 (page 679), we stipulate that each unit has an extra input from a dummy unit 0 that is fixed to +1 and a weight $w_{0,j}$ for that input. This allows the total weighted input in_j to unit j to be nonzero even when the outputs of the preceding layer are all zero. With this convention, we can write the preceding equation in vector form:

$$a_j = g_j(\mathbf{w}^T \mathbf{x}) \quad (21.1)$$

where $\mathbf{w}$ is the vector of weights leading into unit j (including $w_{0,j}$) and $\mathbf{x}$ is the vector of inputs to unit j (including the +1).

A variety of different activation functions are used. The most common are the following:

- The logistic or **sigmoid** function, which is also used in logistic regression (see page 685):

$$\sigma(x) = 1/(1 + e^{-x}).$$

- The **ReLU** function, whose name is an abbreviation for **rectified linear unit**:

$$\text{ReLU}(x) = \max(0, x).$$

- The **softplus** function, a smooth version of the ReLU function:

$$\text{softplus}(x) = \log(1 + e^x).$$

The derivative of the softplus function is the sigmoid function.

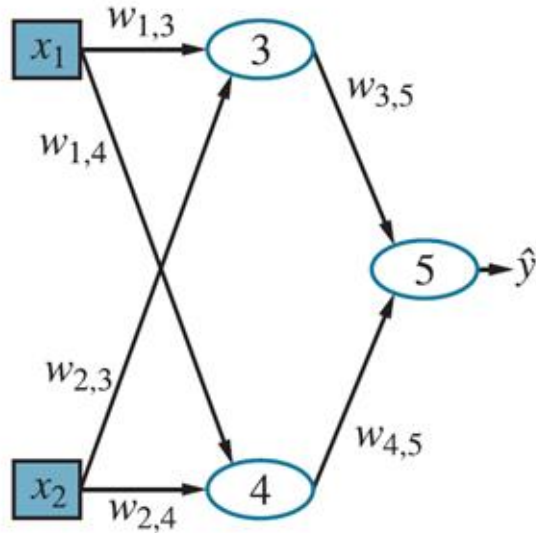
- The **tanh** function:

$$\tanh(x) = \frac{e^{2x}-1}{e^{2x}+1}.$$

Note that the range of tanh is $(-1, +1)$. Tanh is a scaled and shifted version of the sigmoid, as $\tanh(x) = 2\sigma(2x) - 1$.

1. Review

(c) Deep learning in neural networks.



$$\begin{aligned}
 \hat{y} &= g_5(in_5) = g_5(w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4) \\
 &= g_5(w_{0,5} + w_{3,5}g_3(in_3) + w_{4,5}g_4(in_4)) \\
 &= g_5(w_{0,5} + w_{3,5}g_3(w_{0,3} + w_{1,3}x_1 + w_{2,3}x_2) \\
 &\quad + w_{4,5}g_4(w_{0,4} + w_{1,4}x_1 + w_{2,4}x_2)).
 \end{aligned}$$

21.1.2 Gradients and learning

For now, we will use the squared loss function, L_2 , and we will calculate the gradient for the network in Figure 21.3 with respect to a single training example $(\mathbf{x}, y)$. (For multiple examples, the gradient is just the sum of the gradients for the individual examples.) The network outputs a prediction $\hat{y} = h_{\mathbf{w}}(\mathbf{x})$ and the true value is y , so we have

$$Loss(h_{\mathbf{w}}) = L_2(y, h_{\mathbf{w}}(\mathbf{x})) = \|y - h_{\mathbf{w}}(\mathbf{x})\|^2 = (y - \hat{y})^2.$$

To compute the gradient of the loss with respect to the weights, we need the same tools of calculus we used in Chapter 19—principally the **chain rule**, $\partial g(f(x))/\partial x = g'(f(x)) \partial f(x)/\partial x$. We'll start with the easy case: a weight such as $w_{3,5}$ that is connected to the output unit. We operate directly on the network-defining expressions from Equation (21.2):

$$\begin{aligned}
 \frac{\partial}{\partial w_{3,5}} Loss(h_{\mathbf{w}}) &= \frac{\partial}{\partial w_{3,5}} (y - \hat{y})^2 = -2(y - \hat{y}) \frac{\partial \hat{y}}{\partial w_{3,5}} \\
 &= -2(y - \hat{y}) \frac{\partial}{\partial w_{3,5}} g_5(in_5) = -2(y - \hat{y}) g'_5(in_5) \frac{\partial}{\partial w_{3,5}} in_5 \\
 &= -2(y - \hat{y}) g'_5(in_5) \frac{\partial}{\partial w_{3,5}} (w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4) \\
 &= -2(y - \hat{y}) g'_5(in_5) a_3.
 \end{aligned} \tag{21.4}$$

Perform Gradient Descent

$$w_{3,5} = w_{3,5} - \alpha * \text{grad}$$

Function Approximation

- Function approximation constructs **compact representation** of true utility (value) function and Q -function

- Example: Represent evaluation function for chess as a linear function of **features** (or **basis functions**)

$$\hat{U}_{\theta}(s) = \theta_1 f_1(s) + \theta_2 f_2(s) + \dots + \theta_n f_n(s)$$

What is a feature?

- Function approximation uses the n no. of θ parameters to represent a function over a very large number of states
 - RL agent learns θ that best approximate the evaluation (utility) function

Function Approximation

- Function approximation allows **generalization** from small number of states observed in training data to entire state space
 - Example: Backgammon agent learned to play as well as the best human players by observing only $\approx 10^{12}$ states out of 10^{20} states
- Caveat:
 - If n (no. of parameters) is too small, may fail to achieve good approximation

The Idea

Use a **linear function** to approximate U

$$\hat{U}(\mathbf{s}) = \theta_0 + \theta_1 \mathbf{s}_0 + \theta_2 \mathbf{s}_1 + \cdots + \theta_N \mathbf{s}_N$$

Where N is the number of variables of state $\mathbf{s}$

Approximating Monte Carlo Learning

- For MC learning we get a set of training samples

$$((x_1, y_1), u_1), ((x_2, y_2), u_2), \dots, ((x_n, y_n), u_n)$$

Where u_j is the return of the j^{th} example

- This is a supervised learning problem

$$U^\pi(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1}) \right]$$

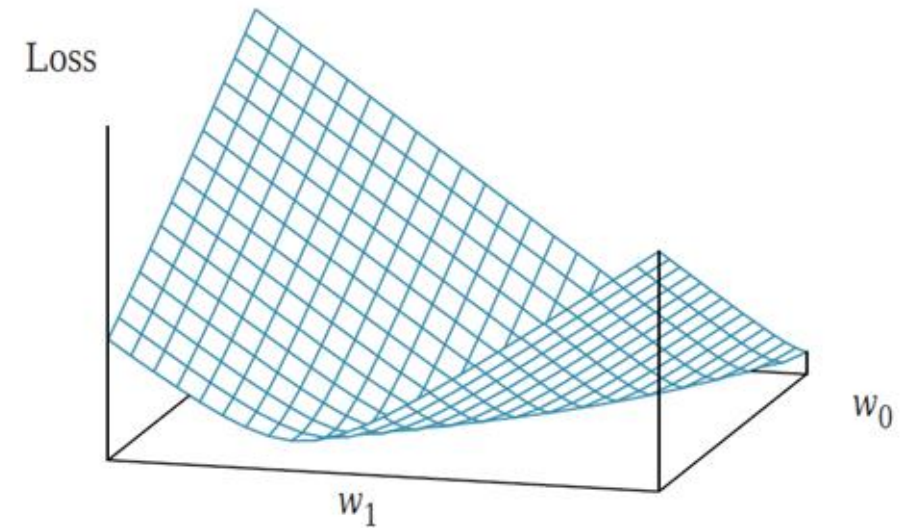
- Standard linear regression problem with mean squared error
 - Error is minimized when partial derivatives wrt to coefficients of linear function are zero

Approximating Monte C

- Learning while interacting with environment
 - Update the parameters after each trial

- Loss function:

- $\mathcal{E}_j(s) = \frac{1}{2} \left(\hat{U}_\theta(s) - u_j(s) \right)^2$
- $u_j(s)$ is the return from state s in the j -th trial



$$((x_1, y_1), u_1), ((x_2, y_2), u_2), \dots, ((x_n, y_n), u_n)$$

$$\hat{U}_\theta(\mathbf{s}) = \theta_0 + \theta_1 \mathbf{s}_0 + \theta_2 \mathbf{s}_1 + \dots + \theta_N \mathbf{s}_N$$

$$\begin{aligned} \frac{\partial \mathcal{E}_j(s)}{\partial \theta_i} &= \frac{\partial \mathcal{E}_j(s)}{\partial \hat{U}_\theta(s)} \frac{\partial \hat{U}_\theta(s)}{\partial \theta_i} \\ &= (\hat{U}_\theta(s) - u_j(s)) \frac{\partial \hat{U}_\theta(s)}{\partial \theta_i} \end{aligned}$$

How do we get the target $u_j(s)$?

Loss Function: $\mathcal{E}_j(s) = \frac{1}{2} \left(\hat{U}_\theta(s) - u_j(s) \right)^2$

Gradient Descent: $\theta_i \leftarrow \theta_i - \alpha \frac{\partial \mathcal{E}_j(s)}{\partial \theta_i}$

$$\begin{aligned} \frac{\partial \mathcal{E}_j(s)}{\partial \theta_i} &= \frac{\partial \mathcal{E}_j(s)}{\partial \hat{U}_\theta(s)} \frac{\partial \hat{U}_\theta(s)}{\partial \theta_i} \\ &= (\hat{U}_\theta(s) - u_j(s)) \frac{\partial \hat{U}_\theta(s)}{\partial \theta_i} \end{aligned}$$

- For utilities:

$$\theta_i \leftarrow \theta_i + \alpha \left[\underbrace{R(s, a, s') + \gamma \hat{U}_\theta(s')}_{\text{TD target}} - \hat{U}_\theta(s) \right] \frac{\partial \hat{U}_\theta(s)}{\partial \theta_i}$$

- For Q -learning:

$$\theta_i \leftarrow \theta_i + \alpha \left[\underbrace{R(s, a, s') + \gamma \max_a \hat{Q}_\theta(s', a)}_{\text{TD target}} - \hat{Q}_\theta(s, a) \right] \frac{\partial \hat{Q}_\theta(s, a)}{\partial \theta_i}$$

- Approximate Q-learning uses a semi-gradient update

$$\delta = r + \gamma \max_{a'} Q^\theta(s', a') - Q^\theta(s, a)$$

$$\theta \leftarrow \theta + \alpha \delta \nabla_\theta Q^\theta(s, a)$$

Different state–action pairs share the same parameter vector θ .

The bootstrap target also depends on θ , but the semi-gradient treats that target as fixed during this update.

Function approximation, bootstrapping, and off-policy learning together can be unstable.

The three ingredients form the deadly triad.



05 Tutorial questions and solutions

Every question restates its data and every answer repeats the update that produced it

RECALL · DISCUSS · CALCULATE

■ Problem 1 · Review

OFFICIAL QUESTION

Briefly explain each concept in your own words.

- (a)** Linear regression and loss-function optimization
- (b)** Gradient descent
- (c)** Deep learning in neural networks

prediction → loss → gradient → parameter update

■ Problem 1 · Regression, optimization, and deep networks

OFFICIAL SOLUTION

Linear regression

$$\hat{y} = \theta^T x$$

$$L(\theta) = (1/N) \sum_i (y_i - \hat{y}_i)^2$$

Training selects θ that minimizes the mean squared error.

Gradient descent

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L(\theta)$$

$\alpha > 0$ is the learning rate. Each step moves parameters in a direction that reduces the loss.

Deep learning composes layers of linear transformations and nonlinear activations.
Forward pass: prediction · Backpropagation: loss gradients · Optimizer: parameter update

AIMA 4e §§19.6.1–19.6.3 and Global Edition §§22.1, 22.2, 22.4

■ Problem 2 · Approximating TD learning

OFFICIAL QUESTION

Navigation state $s=(x,y)$, goal (x_g,y_g) , observed transition (s,r,s') , learning rate α , discount γ .
Write the parameter updates for one-step TD learning.

$$\hat{U}\theta(x,y) = \theta_0 + \theta_1 x + \theta_2 y + \theta_3 \sqrt{[(x-x_g)^2 + (y-y_g)^2]}$$

Discussion prompts

What is the feature vector $\phi(s)$?

What is the one-step TD error δ ?

Which state supplies the gradient?

What value should a terminal successor contribute?

■ Problem 2 · Semi-gradient TD updates

OFFICIAL SOLUTION

$$\hat{U}\theta(x,y) = \theta_0 + \theta_1 x + \theta_2 y + \theta_3 \sqrt{[(x-x_g)^2 + (y-y_g)^2]}$$

$$d(x,y) = \sqrt{[(x-x_g)^2 + (y-y_g)^2]}$$

$$\delta = r + \gamma \hat{U}\theta(s') - \hat{U}\theta(s)$$

$$\theta \leftarrow \theta + \alpha \delta \nabla \hat{U}\theta(s)$$

Terminal successor: set its continuation value to zero, so the TD target becomes r .

$$\theta_i \leftarrow \theta_i + \alpha [R(s, a, s') + \gamma \hat{U}_\theta(s') - \hat{U}_\theta(s)] \frac{\partial \hat{U}_\theta(s)}{\partial \theta_i}$$

■ Problem 3(a) · Two tabular Q-learning updates

OFFICIAL QUESTION

State $x \in \{0, 1\}$; actions a_1, a_2 ; $\gamma = 0.9$; $\alpha = 0.5$; all Q entries initially zero. Show the table after each transition.

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Transition 1

$x=0, r=10, a=a_1, x'=1$

Transition 2

$x=1, r=-5, a=a_2, x'=0$

Carry the updated table from transition 1 into transition 2.

■ Problem 3(a) · After transition 1

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Given $x=0$, $r=10$, $a=a_1$, $x'=1$; $\gamma=0.9$; $\alpha=0.5$; initial $Q=0$.

Q	x=0	x=1
a ₁	-	0
a ₂	0	0

Only the sampled state–action pair changes.

■ Problem 3(a) · After transition 2

OFFICIAL SOLUTION

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_a' Q(s',a') - Q(s,a)]$$

Carry forward $Q(0,a_1)=5$. Now $x=1$, $r=-5$, $a=a_2$, $x'=0$; $\gamma=0.9$; $\alpha=0.5$.

Q	x=0	x=1
a ₁	-	0
a ₂	0	-

The best next-state estimate at $x'=0$ is $\max(5,0)=5$.

■ Problem 3(b) · Replace the Q-table with two parameters

OFFICIAL QUESTION

Use $\alpha=0.5$ and initialize $\beta_1=\beta_2=0$. Show the parameters after each transition.

$$Q(x, a_1) = \beta_1 x \quad Q(x, a_2) = \beta_2 x$$

$$\beta_i \leftarrow \beta_i + \alpha \delta \partial Q(x, a) / \partial \beta_i$$

Transition 1

$x=1, r=10, a=a_1, x'=1$

Notice: transition 1 starts at $x=1$, not $x=0$.

Transition 2

$x=1, r=-5, a=a_2, x'=0$

$$\theta_i \leftarrow \theta_i + \alpha \left[R(s, a, s') + \gamma \max_a \hat{Q}_\theta(s', a') - \hat{Q}_\theta(s, a) \right] \frac{\partial \hat{Q}_\theta(s, a)}{\partial \theta_i}$$

■ Problem 3(b) · After transition 1

OFFICIAL SOLUTION

Given $x=1$, $r=10$, $a=a_1$, $x'=1$; $\gamma=0.9$; $\alpha=0.5$; $\beta_1=\beta_2=0$.

$$Q(x, a_1) = \beta_1 x \quad Q(x, a_2) = \beta_2 x$$

$$\delta = ?$$

$$\partial Q(x, a_1) / \partial \beta_1 = x = 1$$

$$\partial Q(x, a_1) / \partial \beta_2 = 0$$

$$\beta_1 = ? \quad \beta_2 = ?$$

$$\theta_i \leftarrow \theta_i + \alpha \left[R(s, a, s') + \gamma \max_a \hat{Q}_\theta(s', a') - \hat{Q}_\theta(s, a) \right] \frac{\partial \hat{Q}_\theta(s, a)}{\partial \theta_i}$$

■ Problem 3(b) · After transition 2

OFFICIAL SOLUTION

Carry forward $\beta_1=5$, $\beta_2=0$. Now $x=1$, $r=-5$, $a=a_2$, $x'=0$; $\gamma=0.9$; $\alpha=0.5$.

$$Q(x, a_1) = \beta_1 x \quad Q(x, a_2) = \beta_2 x$$

$$\delta = ?$$

$$\beta_1 = ? \quad \beta_2 = ?$$

$$\theta_i \leftarrow \theta_i + \alpha \left[R(s, a, s') + \gamma \max_a \hat{Q}_\theta(s', a') - \hat{Q}_\theta(s, a) \right] \frac{\partial \hat{Q}_\theta(s, a)}{\partial \theta_i}$$

■ Problem 3(c) · Which representation should perform better here?

OFFICIAL QUESTION

Compare the tabular representation with

$$Q(x, a_1) = \beta_1 x \quad Q(x, a_2) = \beta_2 x$$

Discuss

Which representation has more independent degrees of freedom?

What values must the approximator predict when $x=0$?

How could you improve the feature representation?

Does a fixed learning rate $\alpha=0.5$ provide an asymptotic convergence schedule?

■ Problem 3(c) · Representation capacity is the deciding issue

OFFICIAL SOLUTION

For this small problem, the table is better because it assigns an independent value to every state–action pair.

Given model: $Q(0, a_1) = Q(0, a_2) = ?$

It therefore cannot represent arbitrary values at $x=0$.

Improved: $Q(x, a_1) = ?$ $Q(x, a_2) = ?$

Tabular Q-learning converges under the usual finite-MDP conditions, including infinitely many visits and suitable diminishing learning rates. The fixed $\alpha=0.5$ is only for these two illustrative updates.

■ Problem 4(a) · Persy learns with REINFORCE

OFFICIAL QUESTION

Persy observes camera state s_t and samples a drive command $a_t \in \{\text{forward}, \text{left}, \text{right}, \text{stop}\}$ from a stochastic softmax policy $\pi_\theta(a|s)$. The environment returns rewards. REINFORCE learns from completed episodes.

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k$$

Why does REINFORCE use

$$\nabla \theta \log \pi_\theta(a_t|s_t) \cdot G_t$$

Explain the gradient, the return weight, exploration, and credit assignment.

■ Problem 4(a) · The score gradient is weighted by experience

OFFICIAL SOLUTION

$$\nabla \theta \log \pi_{\theta}(a_t | s_t)$$

Direction of greatest local increase in the sampled action's log-probability.

$$G_t$$

Weights that direction using the sampled future return.

$$G_t > 0$$

reinforce

$$G_t = 0$$

no contribution

$$G_t < 0$$

reverse direction

■ Problem 4(b) · Baseline and state value

OFFICIAL QUESTION

A baseline $b(s_t)$ depends on the state but not on the action sampled at that state. REINFORCE may replace G_t by $G_t - b(s_t)$.

G_t

return from one
sampled episode

$V\pi(s_t)$

average return over possible
future trajectories

How does subtracting a baseline help, and why is $b(s_t) = V\pi(s_t)$ a good choice?

■ Problem 4(b) · The baseline reduces variance

OFFICIAL SOLUTION

An action-independent baseline leaves the expected policy gradient unchanged, but it can reduce variance.

$$b(s_t) = V\pi(s_t)$$

$$G_t - V\pi(s_t)$$

??

??

■ Problem 4(c) · Compute the discounted returns

OFFICIAL QUESTION

$$G_t = \sum_{k=t}^{\infty} \gamma^{k-t} r_k$$

Episode ends after three actions

$r_0=1, \quad r_1=0, \quad r_2=2, \quad \gamma=0.9$

Compute G_1 and G_0 .

Work backward from the final reward.

■ Problem 4(c) · Returns from the three-action episode

OFFICIAL SOLUTION

Given $r_0=1$, $r_1=0$, $r_2=2$ and $\gamma=0.9$.

From $t=1$

$$G_1 = r_1 + \gamma r_2 = ??$$

From $t=0$

$$G_0 = r_0 + \gamma r_1 + \gamma^2 r_2 = ??$$

The return-to-go depends on the time step: G_0 includes all three rewards, while G_1 starts at r_1 .

■ Problem 5(a–c) · PPO ratio, advantage, and clipping

OFFICIAL QUESTION

A spacecraft controller is trained with PPO. Here $r_t(\theta)$ is the policy ratio—not the reward r_t from Problem 4.

$$r_t(\theta) = \pi_{\theta}(a_t|s_t) / \pi_{\text{old}}(a_t|s_t)$$

- (a) What does $r_t > 1$, $r_t < 1$, or $r_t = 1$ mean?
- (b) What does the sign of the estimated advantage $\hat{A}_t$ mean?
- (c) Let $\epsilon = 0.2$ and $r_t = 1.3$. Compute $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)$.

State the clipping interval before computing the value.

■ Problem 5(a–c) · Interpret first, then clip

OFFICIAL SOLUTION

Policy ratio

$r_t > 1$: sampled action became more likely

$r_t < 1$: sampled action became less likely

$r_t = 1$: probability is unchanged

Advantage sign

$\hat{A}_t > 0$: better than expected; make action more likely

$\hat{A}_t < 0$: worse than expected; make action less likely

$$[1-\epsilon, 1+\epsilon] = [0.8, 1.2]$$

$$\text{clip}(1.3, 0.8, 1.2) = ??$$

■ Problem 5(d) · Compute the per-sample PPO objective

OFFICIAL QUESTION

Given $r_t=1.3$, $\epsilon=0.2$, $\hat{A}_t=2.0$, and $\text{clip}(r_t, 0.8, 1.2)=1.2$.

Compute

$$u = r_t \hat{A}_t$$

$$c = \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \hat{A}_t$$

$$\mathbf{L_tclip} = \min(u, c)$$

Why does the minimum remain the rule when the advantage is negative?

■ Problem 5(d) · Apply min after multiplying by the advantage

OFFICIAL SOLUTION

$u = ?$

$c = ?$

$L_t \text{clip} = ??$

For a negative advantage, the rule is still $\min(u, c)$. It is not $\max(u, c)$, and it is not the term with smaller absolute value.

■ Problem 5(e) · PPO clipping versus the TRPO constraint

OFFICIAL QUESTION

How does PPO clipping compare with the KL-divergence constraint in TRPO?

Address both parts

What optimization problem does TRPO solve?

What simpler mechanism does PPO use?

Does clipping strictly guarantee a fixed KL bound?

Use precise language: surrogate objective, constraint, and guarantee.

■ Problem 5(e) · Similar motivation, different mechanism

OFFICIAL SOLUTION

TRPO

Explicit constrained optimization

Uses average KL divergence to limit how far the policy changes

PPO

Simpler clipped surrogate objective

Avoids solving the constrained problem directly

PPO clipping does not strictly guarantee that KL divergence remains below a fixed bound.

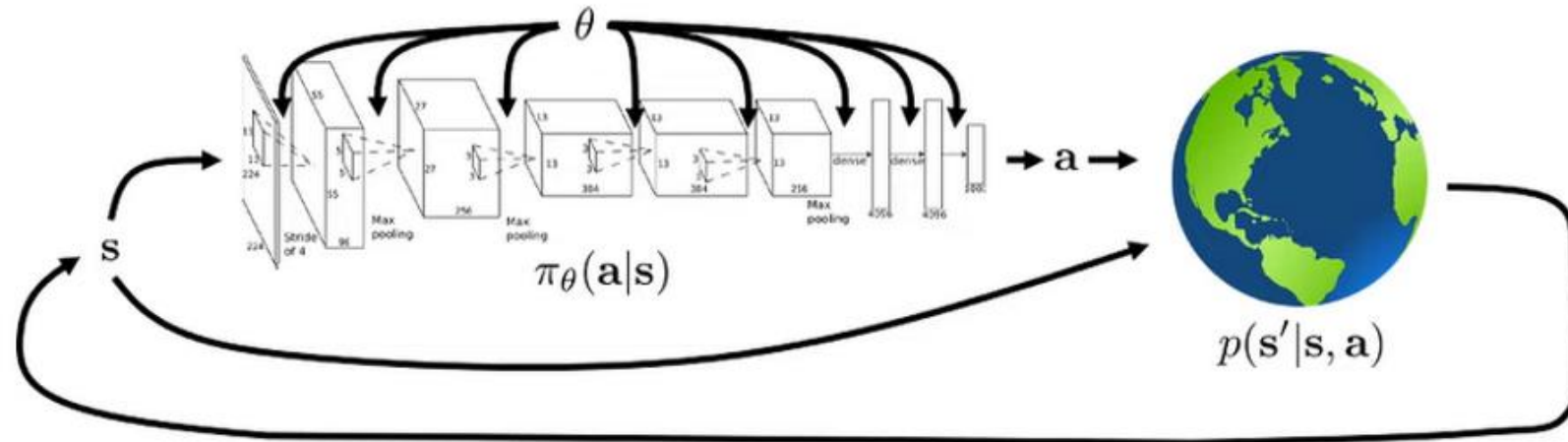


03 Policy search

Optimize a stochastic policy directly, then reduce variance with a learned critic

RECALL · DISCUSS · CALCULATE

Policy Gradients



$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{p_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^T \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\theta^* = \arg \max_{\theta} \underbrace{E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]}_{J(\theta)}$$

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

$$J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right] \approx \frac{1}{N} \sum_i \sum_t r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

Optimization Objective

Policy Gradients

$$\theta^* = \arg \max_{\theta} \underbrace{E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]}_{J(\theta)}$$

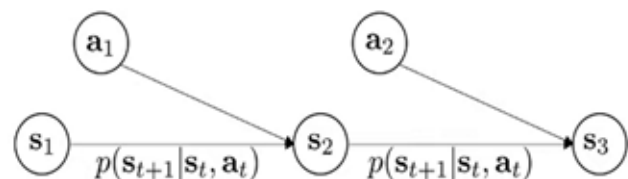
$$J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[\underbrace{r(\tau)}_{\sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t)} \right] = \int p_{\theta}(\tau) r(\tau) d\tau$$

$$\nabla_{\theta} J(\theta) = \int \nabla_{\theta} p_{\theta}(\tau) r(\tau) d\tau = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) r(\tau) d\tau = E_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) r(\tau)]$$

a convenient identity

$$\underline{p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)} = p_{\theta}(\tau) \frac{\nabla_{\theta} p_{\theta}(\tau)}{p_{\theta}(\tau)} = \underline{\nabla_{\theta} p_{\theta}(\tau)}$$

Policy Gradients



$$p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T) = p(\mathbf{s}_1) \prod_{t=1}^T \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$p_{\theta}(\tau)$

log of both sides

$$\log p_{\theta}(\tau) = \log p(\mathbf{s}_1) + \sum_{t=1}^T \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) + \log p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) r(\tau)]$$

$$\nabla_{\theta} \left[\cancel{\log p(\mathbf{s}_1)} + \sum_{t=1}^T \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) + \cancel{\log p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)} \right]$$

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \right) \left(\sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t) \right) \right]$$

Policy Gradients

$$\text{recall: } J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right] \approx \frac{1}{N} \sum_i \sum_t r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \right) \left(\sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t) \right) \right]$$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$$

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

Policy Gradients

Objective function

policy gradient: $J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$

maximum likelihood: $J_{\text{ML}}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right)$

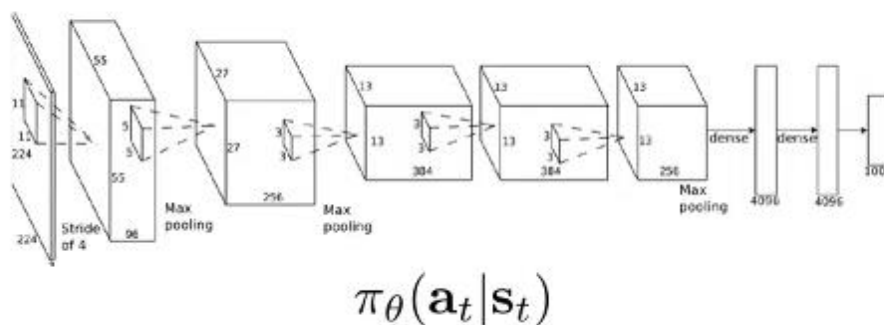
Cross Entropy

Loss
 $L = - \sum_{k=1}^K y_k \log(p_k)$

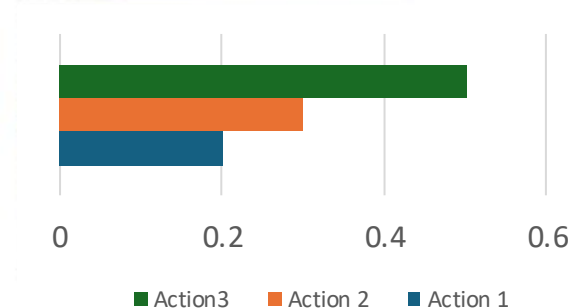
For discrete actions, this is just cross entropy used in supervised learning!



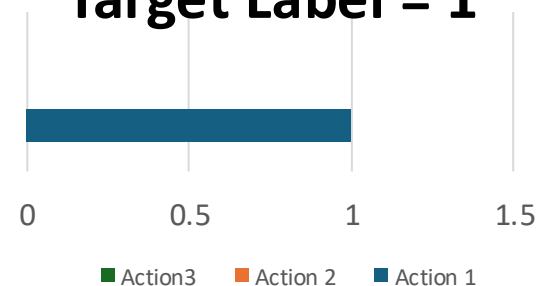
s_t



$\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$



Target Label = 1



**Cross Entropy Loss
= $-\log 0.2$**

Behaviour Cloning

Policy Gradients

Objective function

policy gradient: $J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$

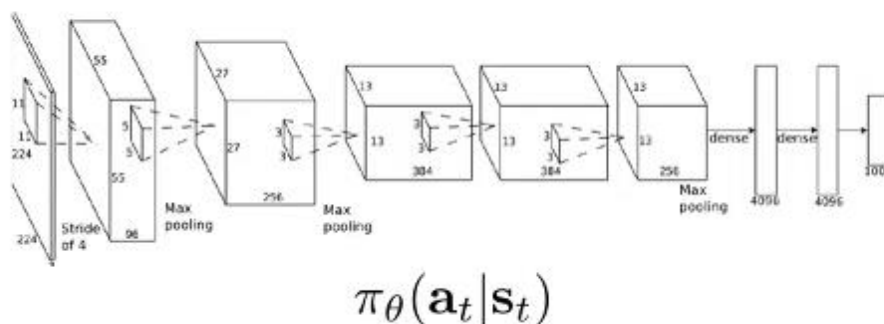
maximum likelihood: $J_{\text{ML}}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right)$

Policy Gradient is BEHAVIOUR CLONING, with action probabilities re-weighted by the sum of rewards-to-go!

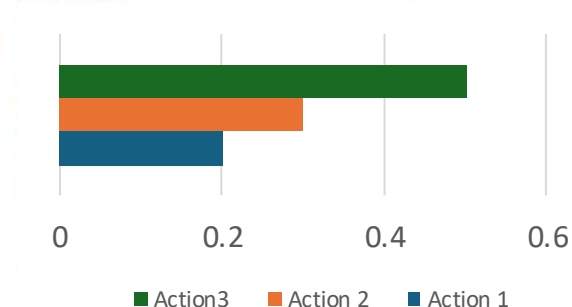
For discrete actions, this is just cross entropy used in supervised learning!



s_t



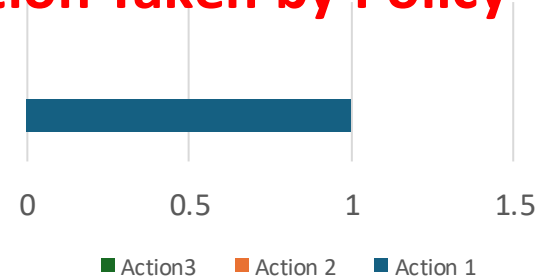
$\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$



Sum Rewards = 2

Cross Entropy Loss = $-\log 0.2$

Action Taken by Policy = 1



Policy Gradient Loss = $-\log 0.2 * 2$

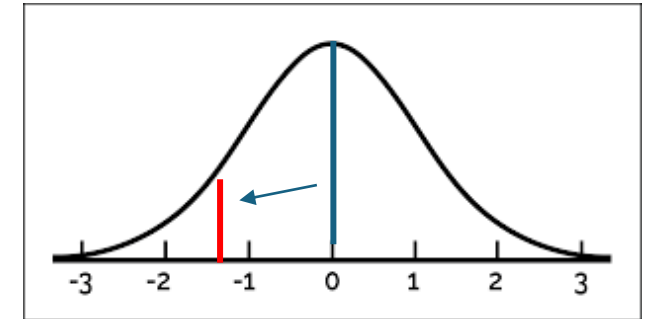
Policy Gradient

Policy Gradients: for continuous actions

Objective function

policy gradient: $J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$

maximum likelihood: $J_{\text{ML}}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right)$

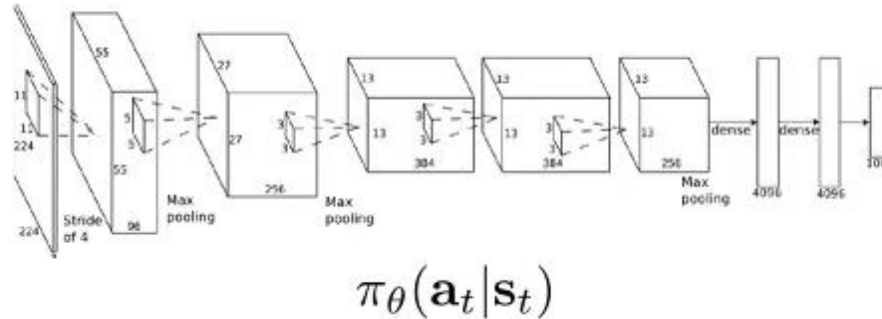


Blue: predicted mean of action
Red: Labelled action in data

MSE between predicted action and labelled action



$\mathbf{s}_t$



$\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$

$$\mathcal{N}(a; \mu_{\theta}, c)$$

Normal Distribution with:
mean = nn output
std = constant

Shifting the mean
towards data
correspond to a MSE

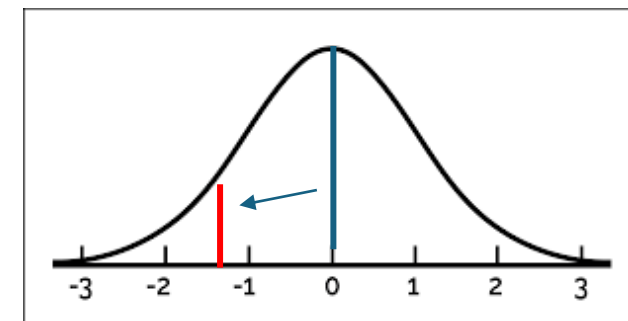
$$f(x) = \frac{1}{\sigma \sqrt{2\pi}} e^{-\frac{1}{2} \left(\frac{x-\mu}{\sigma} \right)^2}$$

Policy Gradients: for continuous actions

Objective function

policy gradient: $J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$

maximum likelihood: $J_{ML}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right)$



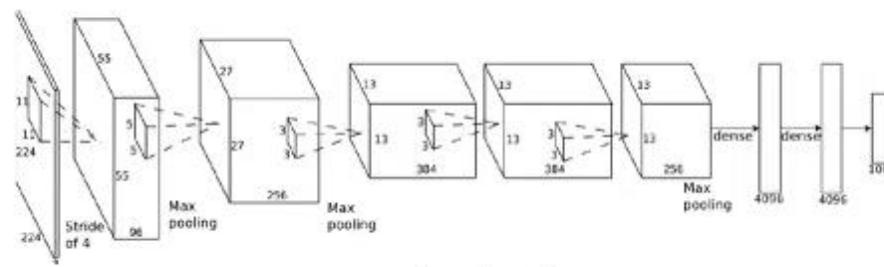
Blue: predicted mean of action

Red: Labelled action in data

MSE between predicted action and labelled action



$\mathbf{s}_t$



$\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$

$$\mathcal{N}(a; \mu_{\theta}, c)$$

Normal Distribution with:
mean = nn output
std = constant

Shifting the mean
towards data
correspond to a MSE

$$\pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) = \frac{1}{c \sqrt{2\pi}} e^{-\frac{1}{2} \left(\frac{\mathbf{a}_{i,t} - \mu_{\theta}}{c} \right)^2}$$

Annotations for the equation:

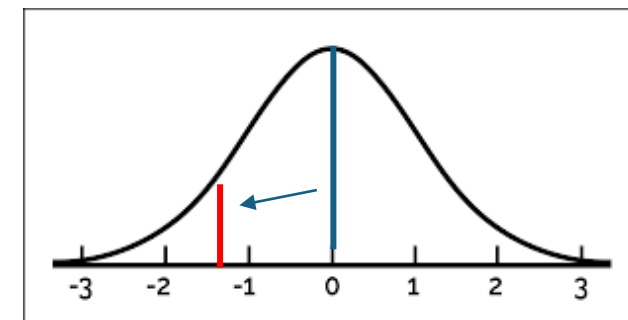
- constants (pointing to c)
- NN Output (pointing to μ_{θ})
- Action label from data/ Sampled action (pointing to $\mathbf{a}_{i,t}$)

Policy Gradients: for continuous actions

Objective function

policy gradient: $J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$

maximum likelihood: $J_{ML}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right)$



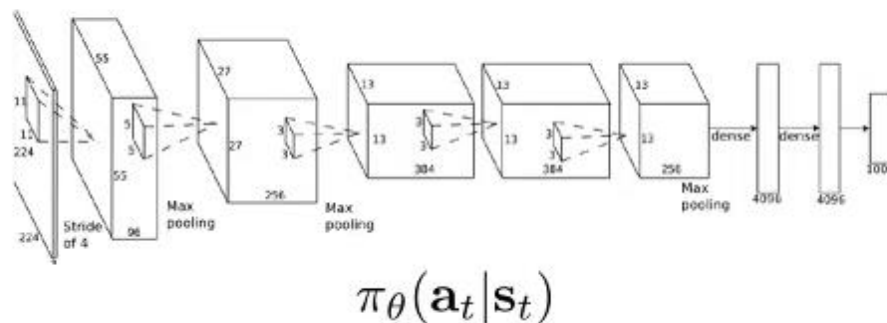
Blue: predicted mean of action

Red: Labelled action in data

MSE between predicted action and labelled action



$\mathbf{s}_t$



$\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$

$$\mathcal{N}(a; \mu_{\theta}, c)$$

Normal Distribution with:
mean = nn output
std = constant

Shifting the mean
towards data
correspond to a MSE

$$\log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t})$$

=

$$-\frac{1}{2} \left(\frac{\mathbf{a}_{i,t} - \mu_{\theta}}{c} \right)^2$$

Action label from data/
Sampled action

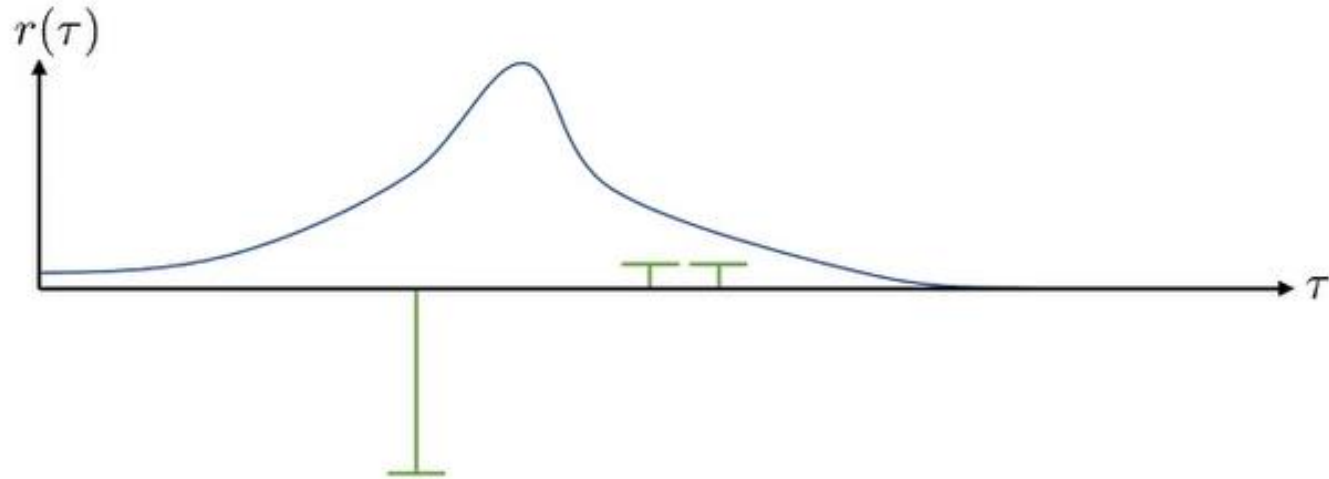
NN Output

Sum Rewards = 2

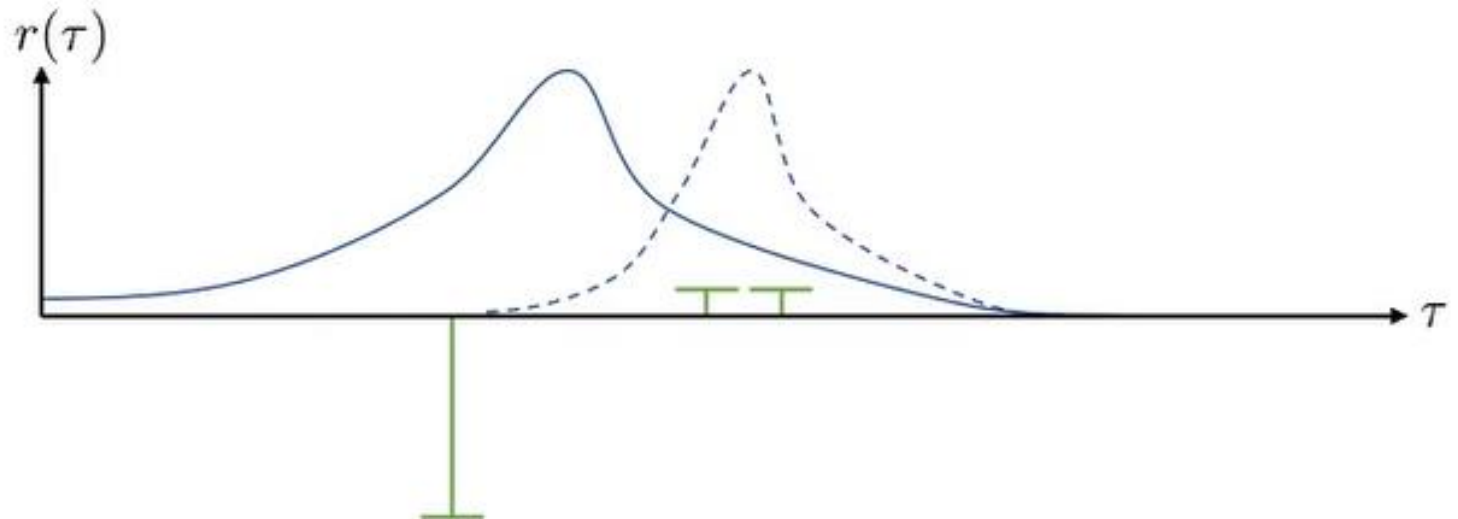
$\mu_{\theta} = 0, a_{\{i,t\}} = -1.3, c = 1$

Actor Critic: Reducing Variance

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log \pi_{\theta}(\tau) r(\tau)$$



$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log \pi_{\theta}(\tau) r(\tau)$$



Actor Critic

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$$



$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \right) \left(Q_{\phi}(s_{i,t}, a_{i,t}) \right)$$

Actor Critic

Can we have a better critic?

Actor Critic: Reducing Variance with Advantage

$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^T E_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t]$: total reward from taking $\mathbf{a}_t$ in $\mathbf{s}_t$

$V^\pi(\mathbf{s}_t) = E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} [Q^\pi(\mathbf{s}_t, \mathbf{a}_t)]$: total reward from $\mathbf{s}_t$

$A^\pi(\mathbf{s}_t, \mathbf{a}_t) = Q^\pi(\mathbf{s}_t, \mathbf{a}_t) - V^\pi(\mathbf{s}_t)$: how much better $\mathbf{a}_t$ is

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) A^\pi(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

What should we train? Q? V? A?

Actor Critic

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) A^{\pi}(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

$$Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + E_{\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)} [V^{\pi}(\mathbf{s}_{t+1})]$$

APPROXIMATION:

assume $\mathbf{s}_{t+1}$ is representative of ALL the $\mathbf{s}_{t+1}$

$$Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t) \approx r(\mathbf{s}_t, \mathbf{a}_t) + V^{\pi}(\mathbf{s}_{t+1})$$

$$A^{\pi}(\mathbf{s}_t, \mathbf{a}_t) \approx r(\mathbf{s}_t, \mathbf{a}_t) + V^{\pi}(\mathbf{s}_{t+1}) - V^{\pi}(\mathbf{s}_t)$$

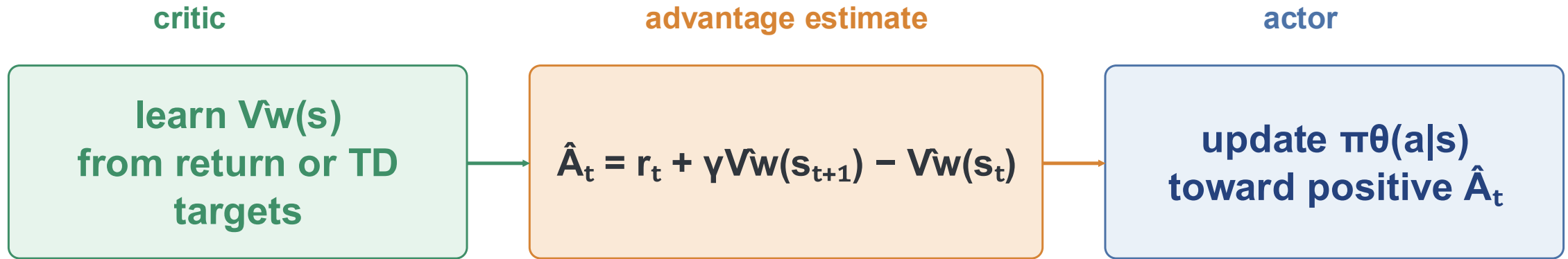
let's just fit $V^{\pi}(\mathbf{s})$!

Learn the Value function with TD!

$$\text{training data: } \left\{ \left(\mathbf{s}_{i,t}, \underbrace{r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) + \hat{V}_{\phi}^{\pi}(\mathbf{s}_{i,t+1})}_{y_{i,t}} \right) \right\}$$

$$\text{supervised regression: } \mathcal{L}(\phi) = \frac{1}{2} \sum_i \left\| \hat{V}_{\phi}^{\pi}(\mathbf{s}_i) - y_i \right\|^2$$

- Actor–critic supplies the advantage signal that PPO needs



$$\nabla\theta J \approx E[\nabla\theta \log \pi\theta(a_t|s_t) \cdot \hat{A}_t]$$

With discounting, keep γ in the TD advantage. The old slide shorthand omitted it.



04 Proximal Policy Optimization

Reuse on-policy data for several updates while clipping incentives for destructive policy changes

RECALL · DISCUSS · CALCULATE

■ PPO compares the updated policy with the behavior policy

The current batch was generated by a fixed behavior policy π_{old} .

$$\rho_t(\theta) = \pi_{\theta}(a_t|s_t) / \pi_{\text{old}}(a_t|s_t)$$

$$\rho_t > 1$$

sampled action became
more likely

$$\rho_t = 1$$

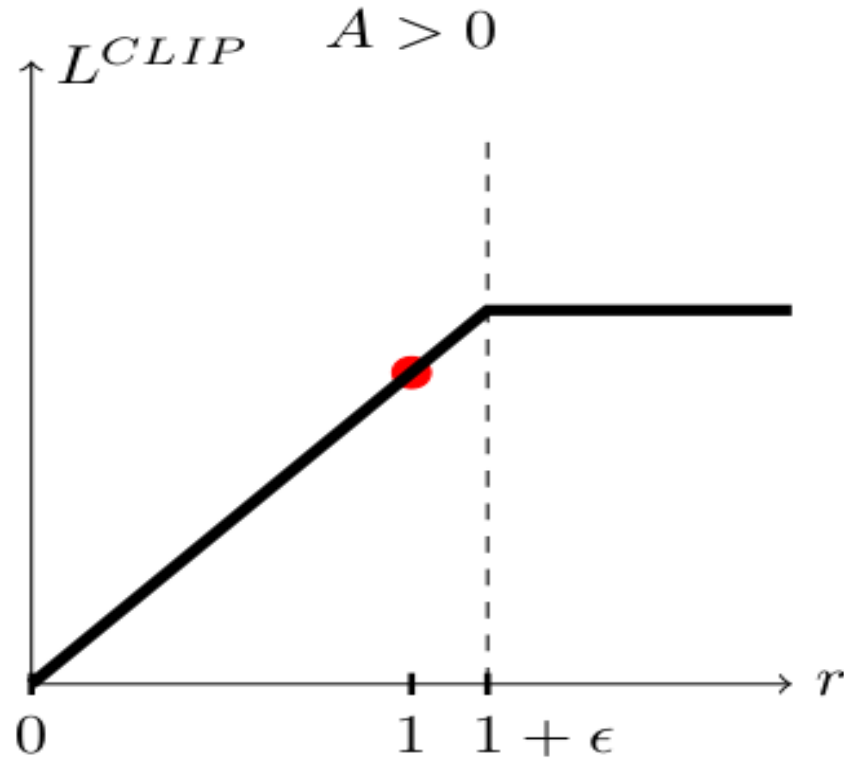
sampled action probability
is unchanged

$$\rho_t < 1$$

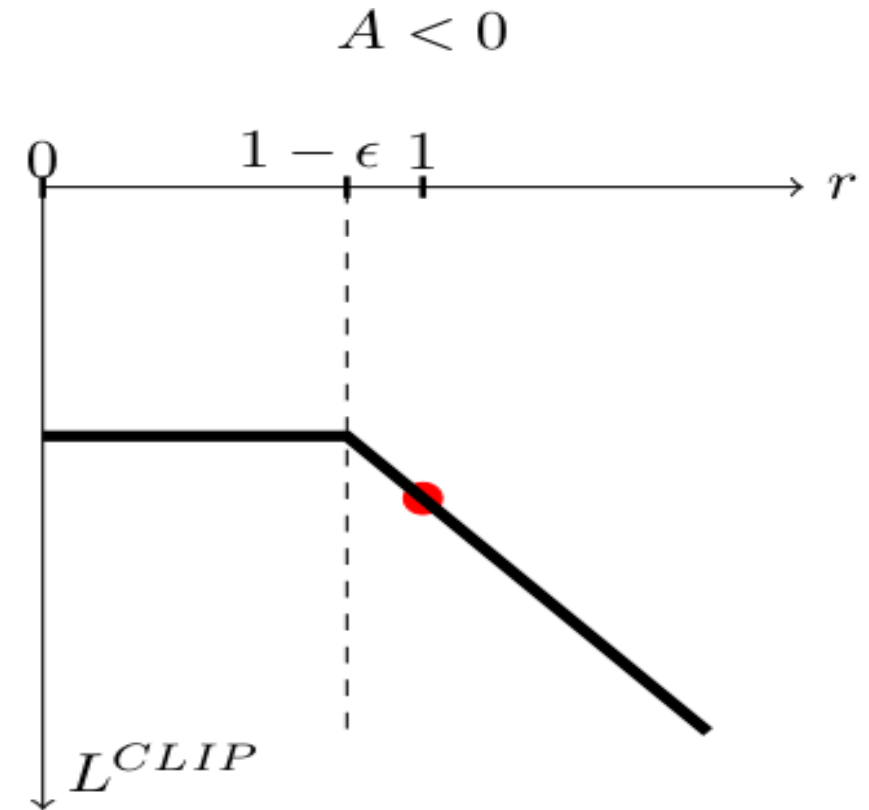
sampled action became
less likely

$$\text{LCLIP}(\theta) = -E_t[\min(\rho_t(\theta)A_t, \text{clip}(\rho_t(\theta_t), 1 - \epsilon, 1 + \epsilon)A_t)]$$

■ Maximising L CLIP



Increasing the probability helps,
but benefit stops beyond $1 + \epsilon$.

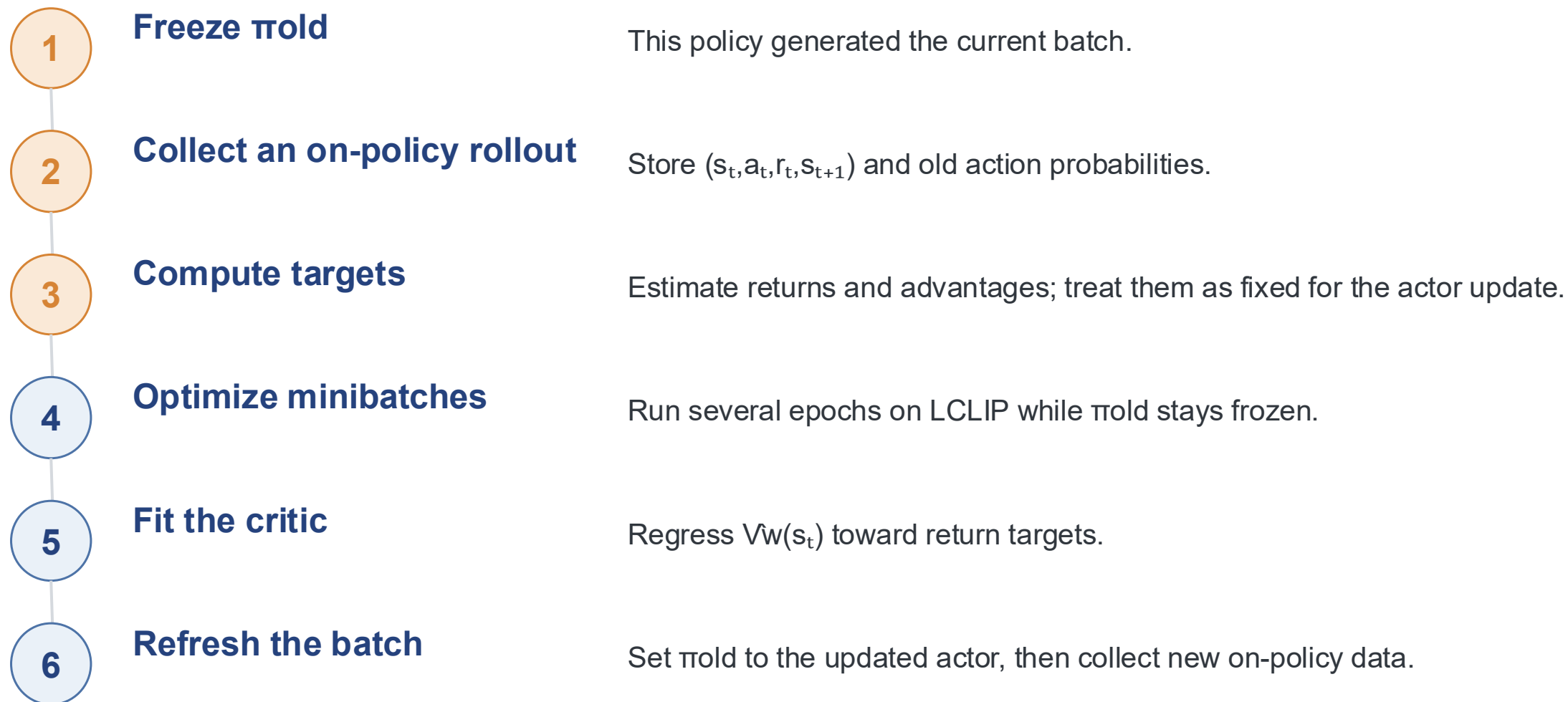


Reducing the probability helps,
but benefit stops below $1 - \epsilon$.

PPO encourages the policy to choose good actions more often and bad actions less often, but gives no extra reward for pushing either change too far.

$$L_{CLIP}(\theta) = E_t[\min(\rho_t(\theta)A_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

■ One practical PPO iteration keeps the data-generating policy fixed



- PPO clipping is a practical surrogate, not a hard trust region

TRPO

**maximize surrogate
subject to a KL constraint**

Explicit constrained optimization

KL divergence defines policy closeness

More complex to implement

PPO

**maximize a clipped
surrogate objective**

Simple first-order objective

Clipping limits incentives for harmful large changes

No strict guarantee that KL stays below a fixed bound

Both methods address the same failure mode: an unreliable policy-gradient step can destroy performance.

- The full chain separates value targets from policy updates

Value-based route

$$\text{Q target} \\ r + \gamma \max_{a'} Q(s', a')$$

fit $Q\theta(s, a)$

Policy-based route

$$\text{advantage estimate} \\ r + \gamma V(s') - V(s)$$

fit $Vw(s)$ and update $\pi\theta(a|s)$

Shared principle: estimate how good the observed choice was, then move parameters in the direction supported by that estimate.